

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(УНИВЕРСИТЕТ ИТМО)

Факультет «Систем управления и робототехники»

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

По дисциплине «Методы оптимизации»

Студенты:
Охрименко Ева
Комарова Ольга

Преподаватель:
преподаватель преподавательевич

г. Санкт-Петербург
2025

Содержание

1 Task. Практическое задание	2
1.1 Реализация методов для численной оценки градиента и гессиана	2
1.1.1 Градиент	2
1.1.2 Гессиан	2
1.2 Оценка зависимости точности градиента и гессиана от выбора шага h	3
1.2.1 Определение функций	3
1.2.2 Функция оценки точности	6
1.2.3 Графики и анализ	7
1.3 Некоторый способ создания функции	8
2 Task. Метод Ньютона	8
2.1 Реализация метода	8
2.2 Тестирование метода на квадратичной функции	9

1 Task. Практическое задание

1.1 Реализация методов для численной оценки градиента и гессиана

1.1.1 Градиент

Формула для каждой компоненты градиента с использованием центральной разностной схемы:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x} - h\mathbf{e}_i)}{2h}$$

Где:

- h - малый шаг (эпсилон)
- f - дифференцируемая функция
- x - точка вычисления
- $\mathbf{e}_i$ - i -й базисный вектор

Весь градиент - это вектор таких производных:

$$\nabla f(\mathbf{x}) \approx \left(\frac{f(x+h, y) - f(x-h, y)}{2h}, \frac{f(x, y+h) - f(x, y-h)}{2h} \right)$$

Вот код, который реализует это:

```
1 def numericalGradient(f, x, h=1e-5):
2     n = len(x)
3     grad = np.zeros_like(x)
4     for i in range(n):
5         e_i = np.zeros_like(x)
6         e_i[i] = h
7         grad[i] = (f(x + e_i) - f(x - e_i)) / (2 * h)
8     return grad
```

Листинг 1: Градиент

1.1.2 Гессиан

Мы хотим найти матрицу вторых производных (гессиан) функции $f(x_1, x_2, \dots, x_n)$ в точке.

Для этого мы:

- Берем маленький шаг h - эпсилон
- Смотрим, как меняется функция при небольших изменениях по разным направлениям
- Используем комбинации значений функции в соседних точках

Мы считаем так:

Для каждой пары переменных (x_i, x_j) :

1. Создаем 4 точки, немного меняя x_i и x_j :

- Точка 1: $x_i + h, x_j + h$

- Точка 2: $x_i + h, x_j - h$
- Точка 3: $x_i - h, x_j + h$
- Точка 4: $x_i - h, x_j - h$

2. Считаем элемент матрицы по формуле:

$$H_{ij} = \frac{f(\text{Точка1}) - f(\text{Точка2}) - f(\text{Точка3}) + f(\text{Точка4})}{4h^2}$$

3. Так как матрица симметричная, $H_{ji} = H_{ij}$
 Для функции $f(x, y)$ гессиан будет:

$$H = \begin{pmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{pmatrix}$$

Вот код:

```

1 def numericalHessian(f, x, h=1e-5):
2     n = len(x)
3     hess = np.zeros((n, n))
4     for i in range(n):
5         for j in range(i, n): x1, x2 = x.copy(), x.copy()
6         x1[i] += h; x1[j] += h
7         x2[i] += h; x2[j] -= h
8         x3, x4 = x.copy(), x.copy()
9         x3[i] -= h; x3[j] += h
10        x4[i] -= h; x4[j] -= h
11        hess[i, j] = (f(x1) - f(x2) - f(x3) + f(x4)) / (4 * h**2)
12        hess[j, i] = hess[i, j]
13
14    return hess

```

Листинг 2: Гессиан

1.2 Оценка зависимости точности градиента и гессиана от выбора шага h

1.2.1 Определение функций

Для начала зададим функции, которые предложены в задании:

- Комбинация из пары тригонометрических функций

Возьмем такую функцию:

$$f(x_1, x_2) = \sin(x_1) \cos(x_2) + \sin(x_1 + x_2)$$

Вот ее реализация:

```

1 def trigFunc(x):
2     return sin(x[0]) * cos(x[1]) + sin(x[0] + x[1])

```

Листинг 3: Тригонометрическая функция

Ее градиент:

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{pmatrix} = \begin{pmatrix} \cos x_1 \cos x_2 + \cos(x_1 + x_2) \\ -\sin x_1 \sin x_2 + \cos(x_1 + x_2) \end{pmatrix}$$

Его реализация:

```
1 def exactGradTrig(x):
2     return np.array([
3         cos(x[0])*cos(x[1]) + cos(x[0] + x[1]),
4         -sin(x[0])*sin(x[1]) + cos(x[0] + x[1])
5     ])
```

Листинг 4: Градиент тригонометрической функции

Также рассмотрим гессиан:

$$\nabla^2 f = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} \end{pmatrix} = \begin{pmatrix} -\sin x_1 \cos x_2 - \sin(x_1 + x_2) & -\cos x_1 \sin x_2 - \sin(x_1 + x_2) \\ -\cos x_1 \sin x_2 - \sin(x_1 + x_2) & -\sin x_1 \cos x_2 - \sin(x_1 + x_2) \end{pmatrix}$$

Реализация кодом:

```
1 def exactHessTrig(x):
2     return np.array([
3         [-sin(x[0])*cos(x[1]) - sin(x[0] + x[1]), -cos(x[0])*sin(x[1]) - sin(x
4         [0] + x[1])],
5         [-cos(x[0])*sin(x[1]) - sin(x[0] + x[1]), -sin(x[0])*cos(x[1]) - sin(x
6         [0] + x[1])]
```

Листинг 5: Гессиан тригонометрической функции

- Полином нескольких переменных степени 3

Вот такую функцию мы выбрали:

$$f(x_1, x_2, x_3) = x_1^3 + 2x_1^2x_2 - 3x_1x_2^2 + x_2^3 + x_1x_2x_3$$

Реализация на питоне:

```
1 def polyFunc(x):
2     return x[0]**3 + 2*x[0]**2*x[1] - 3*x[0]*x[1]**2 + x[1]**3 + x[0]*x
3         [1]*x[2]
```

Листинг 6: Полином нескольких переменных

Также рассчитаем градиент функции и ее гессиан.

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \frac{\partial f}{\partial x_3} \end{pmatrix} = \begin{pmatrix} 3x_1^2 + 4x_1x_2 - 3x_2^2 + x_2x_3 \\ 2x_1^2 - 6x_1x_2 + 3x_2^2 + x_1x_3 \\ x_1x_2 \end{pmatrix}$$

$$\nabla^2 f = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \frac{\partial^2 f}{\partial x_1 \partial x_3} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \frac{\partial^2 f}{\partial x_2 \partial x_3} \\ \frac{\partial^2 f}{\partial x_3 \partial x_1} & \frac{\partial^2 f}{\partial x_3 \partial x_2} & \frac{\partial^2 f}{\partial x_3^2} \end{pmatrix} = \begin{pmatrix} 6x_1 + 4x_2 & 4x_1 - 6x_2 & x_2 \\ 4x_1 - 6x_2 & -6x_1 + 6x_2 & x_1 \\ x_2 & x_1 & 0 \end{pmatrix}$$

Вот код на питоне для градиента и гессиана:

```

1 def exactGradPoly(x):
2     return np.array([
3         3*x[0]**2 + 4*x[0]*x[1] - 3*x[1]**2 + x[1]*x[2],
4         2*x[0]**2 - 6*x[0]*x[1] + 3*x[1]**2 + x[0]*x[2],
5         x[0]*x[1]
6     ])
7
8 def exactHessPoly(x):
9     return np.array([
10        [6*x[0] + 4*x[1], 4*x[0] - 6*x[1], x[1]],
11        [4*x[0] - 6*x[1], -6*x[0] + 6*x[1], x[0]],
12        [x[1], x[0], 0]
13    ])

```

Листинг 7: Градиент и гессиан для полинома нескольких переменных

- Композиция, содержащая $\exp$, $\ln$, $\arctan$ и возведение в степень

Мы взяли такую функцию:

$$f(x_1, x_2, x_3) = e^{x_1} \ln(1 + x_2^2) + \arctg(x_1 x_3)$$

Реализация:

```

1 def complexFunc(x):
2     return exp(x[0]) * log(1 + x[1]**2) + atan(x[0]*x[2])

```

Листинг 8: Композиция нескольких функций

Также рассчитаем градиент и гессиан для этой функции:

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \frac{\partial f}{\partial x_3} \end{pmatrix} = \begin{pmatrix} \frac{z}{x^2 z^2 + 1} + e^x \ln(y^2 + 1) \\ 2y \frac{e^x}{y^2 + 1} \\ \frac{x}{x^2 z^2 + 1} \end{pmatrix}$$

$$\nabla^2 f = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \frac{\partial^2 f}{\partial x_1 \partial x_3} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \frac{\partial^2 f}{\partial x_2 \partial x_3} \\ \frac{\partial^2 f}{\partial x_3 \partial x_1} & \frac{\partial^2 f}{\partial x_3 \partial x_2} & \frac{\partial^2 f}{\partial x_3^2} \end{pmatrix} = \begin{pmatrix} -\frac{2xz^3}{(x^2 z^2 + 1)^2} + e^x \ln(y^2 + 1) & \frac{2ye^x}{y^2 + 1} & \frac{1 - x^2 z^2}{(x^2 z^2 + 1)^2} \\ \frac{2ye^x}{y^2 + 1} & \frac{2e^x(1 - y^2)}{(y^2 + 1)^2} & 0 \\ \frac{1 - x^2 z^2}{(x^2 z^2 + 1)^2} & 0 & -\frac{2x^3 z}{(x^2 z^2 + 1)^2} \end{pmatrix}$$

И также код:

```

1 def exactGradComplex(x):
2     return np.array([
3         exp(x[0])*log(1 + x[1]**2) + x[2]/(1 + (x[0]*x[2])**2),
4         exp(x[0])*(2*x[1])/(1 + x[1]**2),
5         x[0]/(1 + (x[0]*x[2])**2)
6     ])
7
8 def exactHessComplex(x):
9     h11 = exp(x[0])*log(1 + x[1]**2) - (2*x[0]*x[2]**2)/(1 + (x[0]*x[2])
10        **2)**2
11     h12 = exp(x[0])*(2*x[1])/(1 + x[1]**2)
12     h13 = (1 - (x[0]*x[2])**2)/(1 + (x[0]*x[2])**2)**2
13     h22 = exp(x[0])*(2 - 2*x[1]**2)/(1 + x[1]**2)**2
14     return np.array([
15        [h11, h12, h13],

```

```

15 [h12, h22, 0],
16 [h13, 0, -2*x[0]**2*x[2]/(1 + (x[0]*x[2])**2)**2]
17 ]

```

Листинг 9: Градиент и гессиан композиции нескольких функций

1.2.2 Функция оценки точности

Итак, для начала выберем 3 точки, в которых будем все оценивать:

$$\begin{aligned} \mathbf{x}_1 &= (0.5 \quad 1.0 \quad -0.5), \\ \mathbf{x}_2 &= (1.0 \quad -1.0 \quad 2.0), \\ \mathbf{x}_3 &= (0.0 \quad 0.5 \quad 2.5). \end{aligned}$$

```

1 testPoints = [
2     np.array([0.5, 1.0, -0.5]),
3     np.array([1.0, -1.0, 2.0]),
4     np.array([0.0, 0.5, 2.5])
5 ]

```

Листинг 10: Выбор точек

Далее создадим переменную `hValues` и запишем туда 50 чисел:

```

1 hValues = np.logspace(-10, -1, 50)

```

Листинг 11: `hValues` в логарифмическом масштабе

Создаёт 50 чисел, равномерно распределённых в логарифмическом масштабе между 10^{-10} и 10^{-1} .

Можно понять это вот так:

$$h_i = 10^{x_i}, \quad x_i \in [-10, -1]$$

Теперь точно приступим к функции оценки точности:

```

1 def evaluateAccuracy():
2     results = defaultdict(lambda: ([[] for _ in range(3)], [[] for _ in range
3         (3)])) # (grad_errors, hess_errors)
4
5     functions = {
6         'trig': (trigFunc, exactGradTrig, exactHessTrig, 2),
7         'poly': (polyFunc, exactGradPoly, exactHessPoly, 3),
8         'complex': (complexFunc, exactGradComplex, exactHessComplex, 3)
9     }
10
11     for name, (func, exactGrad, exactHess, dim) in functions.items():
12         for i, x_full in enumerate(testPoints):
13             x = x_full[:dim]
14             gradErrors, hessErrors = results[name]
15
16             for h in hValues:
17                 numGrad = numericalGradient(func, x, h)
18                 numHess = numericalHessian(func, x, h)
19
20                 gradErrors[i].append(np.linalg.norm(numGrad - exactGrad(x)))
21                 hessErrors[i].append(np.linalg.norm(numHess - exactHess(x)))
22
23     return results

```

Листинг 12: `evaluateAccuracy()`

Теперь не кратко по ней пройдемся:

- Переменная **results** во второй строчке нужна для инициализации "как бы таблицы" для хранения ошибок. Вот как она выглядит:

Таблица 1: Пример данных в переменной **results**

Функция	Тип ошибки	Точка 1	Точка 2
trig	Градиент	$[1.2e-5, 5.7e-6, \dots]$	$[8.3e-6, 3.1e-6, \dots]$
	Гессиан	$[4.2e-3, 2.1e-3, \dots]$	$[3.8e-3, 1.9e-3, \dots]$
poly	Градиент	$[6.5e-7, 3.2e-7, \dots]$	$[5.1e-7, 2.8e-7, \dots]$
	Гессиан	$[9.4e-4, 4.7e-4, \dots]$	$[8.2e-4, 4.1e-4, \dots]$
complex	Градиент	$[2.3e-5, 1.1e-5, \dots]$	$[1.8e-5, 9.5e-6, \dots]$
	Гессиан	$[7.1e-3, 3.6e-3, \dots]$	$[6.4e-3, 3.2e-3, \dots]$

- Словарь **functions** нужен, чтобы там хранились ссылки на имена функций, аналитически вычисленных в пункте 1.2.1 градиентов и гессианов для этих функций, а также сюда мы записали размерности функций.
- `for name, (func, exactGrad, exactHess, dim) in functions.items():` - тут мы проходимся непосредственно по словарю из предыдущего пункта.
- Далее мы проходимся по нашим **testPoints** и обрезаем точку до размерности функции **dim**, это нужно так как у нас есть 2D и 3D функции. Также в этом **for** мы достаем списки для хранения ошибок.
- `for h in hValues:` - это цикл по всем шагам **hValues**. Тут мы рассчитываем градиент и гессиан в какой-то точке, причем градиент и гессиан берутся уже рассчитанные и зависящие от названия функции + тут же мы рассчитываем численно написанные градиент и гессиан. Далее мы заполняем массивы норм ошибок.

1.2.3 Графики и анализ

Далее мы прописали функцию для отрисовки графиков. Особого смысла приводить код для этого мы не видим, но в конце отчета обязательно приложим ссылку на GitHub со всеми кодами.

Итак, вот такие графики у нас получились:

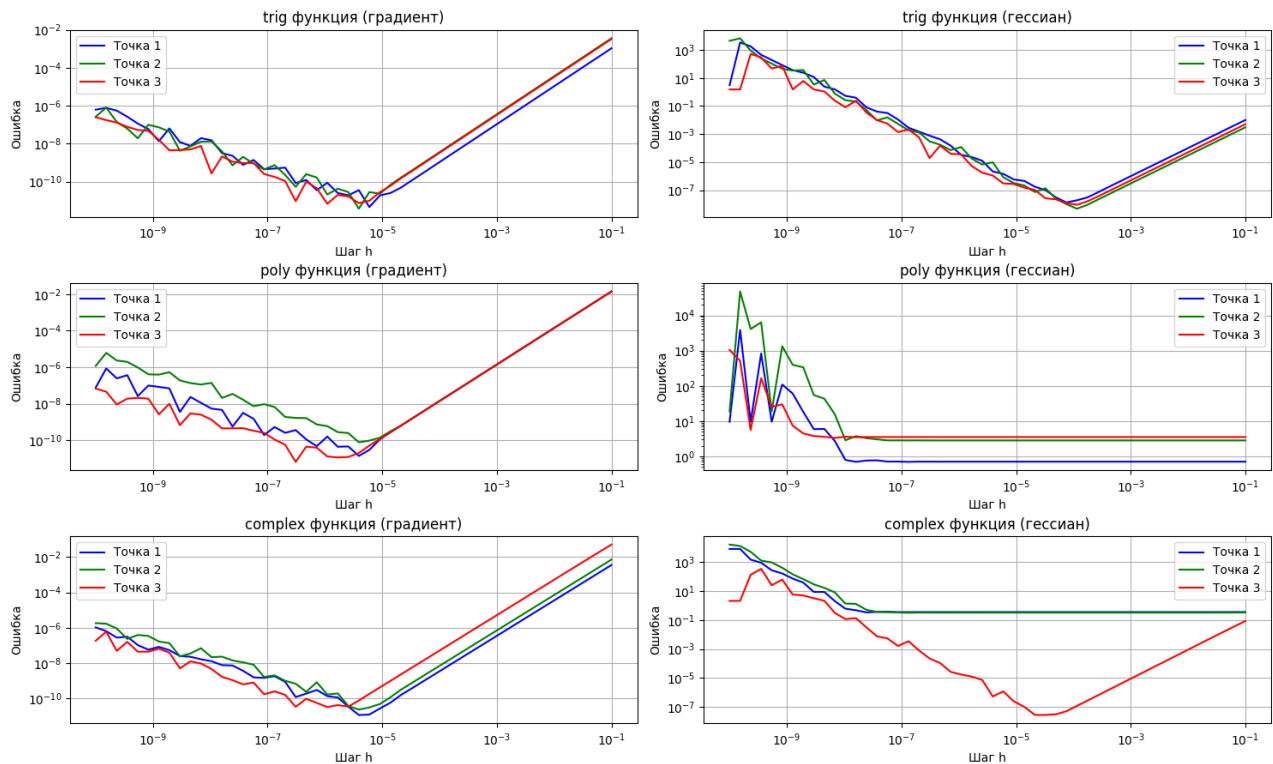


Рис. 1: Графики

По этим графикам можно сказать, что ...

1.3 Некоторый способ создания функции

Оля, тут нужна твоя математическая подкованность

2 Task. Метод Ньютона

Хотим найти минимум функции $f(x)$. На каждой итерации:

$$x_{k+1} = x_k - \alpha(\nabla^2 f(x_k))^{-1} \nabla f(x_k),$$

где:

- $\nabla f(x_k)$ — градиент
- $H(x_k)$ — гессиан
- α — шаг (можно брать 1 или подбирать по стратегии) Архимо-Вульфа

2.1 Реализация метода

- Вычисляем градиент ∇f и гессиан $\nabla^2 f$ численно
- Находим направление d из уравнения $\nabla^2 f \cdot d = -\nabla f$
(в коде: `direction = -np.linalg.solve(hessian, grad)`)
- Выбираем шаг α :

– Если `alpha='wolfe'`, вычисляем α через `arhim`

- Иначе $\alpha = 1$ (чистый метод Ньютона)
- Обновляем точку: $x_{k+1} = x_k + \alpha \cdot d$
- Повторяем, пока $\|\nabla f\| < \text{tol}$

Итак, вот наша реализация этого метода на python:

```

1 def newtonMethod(f, x0, alpha='wolfe', tol=1e-6, maxIter=100, h=1e-5):
2     x = np.array(x0, dtype=float)
3     history = [x.copy()]
4
5     for _ in range(maxIter):
6         grad = numericalGradient(f, x, h)
7         if np.linalg.norm(grad) < tol:
8             break
9
10        hessian = numericalHessian(f, x, h)
11        direction = -np.linalg.solve(hessian, grad)
12
13        if alpha == 'wolfe':
14            step = arhim(f, lambda x: numericalGradient(f, x, h), x, direction)
15        else:
16            step = 1.0
17
18        x += step * direction
19        history.append(x.copy())
20
21    return x, np.array(history)
22

```

Листинг 13: Метод Ньютона

2.2 Тестирование метода на квадратичной функции

В задании сказано, что нужно взять квадратичные функции из прошлой лабораторной. Мы так и сделаем

$$f(\mathbf{x}) = \mathbf{x}^T A \mathbf{x} + \mathbf{b}^T \mathbf{x}$$

$$\mathbf{x}^T A \mathbf{x} + \mathbf{b}^T \mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = 2x_1^2 + 2x_1x_2 + 3x_2^2 + x_1 - x_2$$

```

1 def f(x):
2     A = np.array([[2, 1],
3                   [1, 3]])
4     b = np.array([1, -1])
5
6     f = lambda x: x.T @ A @ x + b.T @ x
7

```

Листинг 14: Задание функции

Далее мы будем исследовать метод Ньютона на этой функции из точки $x_0 = \begin{bmatrix} 1000 \\ 1100 \end{bmatrix}$

Такие графики у нас получились:

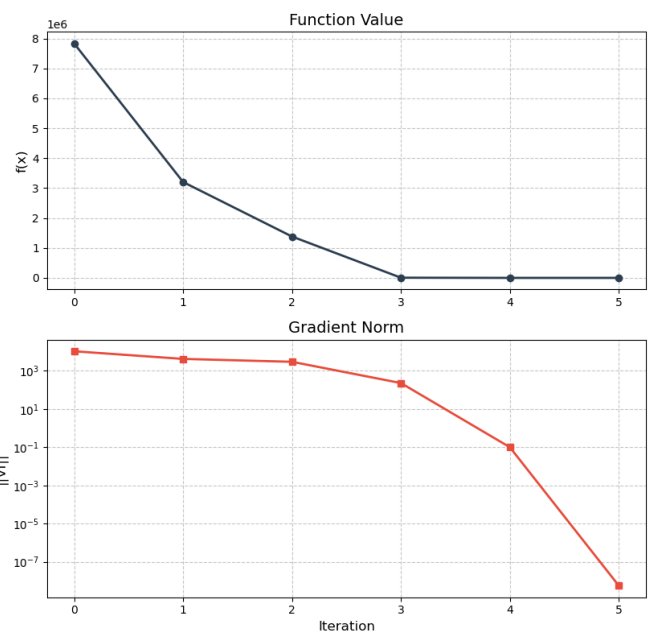
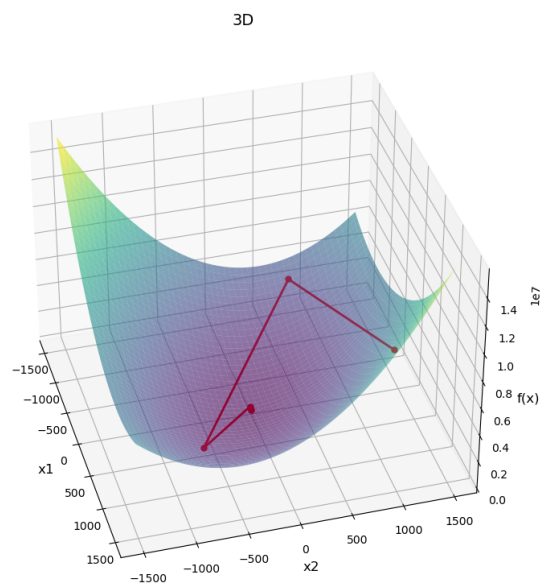


Рис. 2: Графики